Multi-Agent Assisted Automatic Test Generation for Java JSON Libraries

Sinan Wang*
Research Institute of Trustworthy Autonomous Systems
Southern University of Science and Technology
Shenzhen, Guangdong, China
wangsn@mail.sustech.edu.cn

Zhiyuan Zhong*,[1]
School of Computing
University of Utah
Salt Lake City, UT, USA
zhiyuan.zhong@utah.edu

Shaojin Wen
Alibaba Group
Hangzhou, Zhejiang, China
shaojin.wensj@alibaba-inc.com

Yepang Liu[2]
Department of Computer Science and Engineering
Southern University of Science and Technology
Shenzhen, Guangdong, China
liuyp1@sustech.edu.cn

2025 32nd Asia-Pacific Software Engineering Conference (APSEC) | 979-8-3315-6653-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/APSEC66846.2025.00064

***Abstract*—JSON is a widely used data format for data exchange between application systems and programming frontends. In the Java ecosystem, *Java JSON libraries* serve as fundamental toolkits for processing JSON data, powering a wide range of real-world applications, such as web services, Android apps, or data management systems. However, without effective quality assurance methods, such as automatic test generation (ATG), developers risk introducing subtle data inconsistency bugs, compatibility issues, and even security vulnerabilities. These flaws can affect billions of end users and potentially cause severe financial losses.**

Recently, large language models (LLMs) have shown strong potential in enhancing ATG. However, existing LLM-based methods, such as TitanFuzz or YanHui, lack specialization in the JSON domain. For Java JSON libraries, effective bug-triggering test cases should capture the constraints between formatted data and application programs, posing challenges to existing methods, leaving critical aspects of quality assurance unaddressed.

To fill this gap, we propose JSONATG, a multi-agent ATG system that generates diverse bug-triggering tests for Java JSON libraries. With historical bug-triggering unit tests as seeds, JSONATG introduces a code summarization agent and a test validation agent into the generation pipeline to produce new, valid test cases. It applies agent-generated program mutation rules tailored specifically for the structural and semantic characteristics of Java JSON libraries, such as data streaming operations, serialization formats, and data-binding patterns. The generated tests are further refined through post-processing to ensure both syntactic and semantic correctness. Our experiments show that JSONATG achieves higher coverage than two state-of-the-art LLM-based test generation methods on the widely used Java JSON library. Using JSONATG with a $25 budget, we reported 59 bugs (including non-crashing functional bugs) in *fastjson*, of which 47 were confirmed and 28 have already been fixed.

***Index Terms*—Java JSON Library, Automatic Test Generation, Multi-agent System, Program Mutation**

*The first two authors contributed equally to this work.

[1]Work conducted during Zhiyuan Zhong's undergraduate studies at Southern University of Science and Technology.

[2]Yepang Liu is the corresponding author. He is also affiliated with the Research Institute of Trustworthy Autonomous Systems.

I. Introduction

Data-serialization libraries are crucial tools in modern software development. They are responsible for conversion between programmable in-memory objects and data persistence formats, enabling efficient data exchange, storage, and retrieval for application programs. Well-known data-serialization formats include XML, JSON, Protocol Buffers, and so on. In the Java ecosystem, developers widely use JSON in diverse application scenarios, including web services, mobile apps, data management, and scientific computing [1]. We refer to the utilized programming toolkit as *Java JSON library*, which offers essential functionalities like parsing JSON into Java objects, serializing objects into JSON format, and manipulating JSON data. Despite their widespread use, Java JSON libraries can suffer from serious bugs, including performance downgrading [2] and even security threats [3], potentially affecting the reliability and safety of the dependent applications.

The research community has actively explored automated approaches to ensure the quality of such third-party libraries. Fuzzing [4] is a powerful technique that uncovers bugs by generating diverse inputs to exercise software behaviors. However, traditional fuzzing methods often fall short in detecting non-crashing functional bugs that lead to incorrect outputs without triggering unhandled exceptions. Moreover, the generated inputs are typically not reproducible or maintainable, which limits their long-term utility in regression testing. Unit testing allows developers to provide human-written test cases to ensure the correct implementation of the software under test. Researchers have also proposed to automatically generate such tests [5], [6], which we refer to as *automatic test generation* (ATG). However, these techniques cannot be directly applied to Java JSON libraries. A key limitation is their lack of domain knowledge, such as evaluating the correctness of various serialization configurations or understanding the mappings between JSON data and its corresponding schema (i.e., the Java bean class). As a result, their ability to expose subtle

979-8-3315-6653-1/25/$31.00 ©2025 IEEE
DOI 10.1109/APSEC66846.2025.00064

misbehaviors in Java JSON libraries is greatly constrained.

Recent advances in large language model (LLM) technology have shown promise in ATG [7], [8], [9], [10], [11]. For example, TITANFUZZ [12] synthesizes test programs for deep learning libraries with two types of LLM: a generative LLM for creating seed programs and an infilling LLM for rewriting existing programs. YANHUI [13] is a directed ATG approach toward the model optimization module in deep learning libraries. Yet, little work has utilized LLM for addressing the aforementioned challenge in testing Java JSON libraries, leaving the potential of LLM in this domain unexplored.

To bridge this gap, we present JSONATG, a multi-agent ATG system tailored to Java JSON libraries. JSONATG leverages historical bug-triggering test cases and incorporates LLM-based agents for code summarization and test validation. It generates diverse and semantically rich test cases using agent-generated program mutation rules that capture the structural and behavioral characteristics of JSON processing and data-binding in Java. Our experiments show that JSONATG achieves the state-of-the-art performance in generating bug-triggering test suites for popular Java JSON libraries. In summary, this work makes the following contributions:

- We decompose the overall ATG process for Java JSON libraries into three well-defined subtasks and assign each to a dedicated agent, which emulates the workflow of expert human testers in a specialized domain.
- We implement our proposed multi-agent approach as an ATG system, JSONATG, and experimentally show that it outperforms two state-of-the-art LLM-based ATG methods in terms of code coverage and detected bugs.
- With JSONATG, we have reported 59 bugs (including non-crashing functional bugs) in *fastjson*, a popular Java JSON library, with a gross budget of $25. Among the reported bugs, 47 were confirmed and 28 have already been fixed.

II. BACKGROUND

A. Large Language Models for Code

Recent developments have demonstrated the effectiveness of LLMs in code-related tasks, including program synthesis [14], repair [15], debugging [16], fuzzing [17], test improvement [18], and decompilation [19]. Their performance is enhanced with prompt engineering and fine-tuning techniques. Few-shot prompting [20], using curated examples, guides LLMs in solving tasks with desired output. Chain-of-Thought (CoT) prompting [21] helps to break down complex problems into smaller steps. Embedding relevant code snippets, documentation, and instructions leverages models' in-context learning ability for diverse coding scenarios. Fine-tuning adjusts the model weights to optimize performance in specific tasks, but is known to be a resource-intensive approach [22]. The LLM-based multi-agent system (LMA) extends the capabilities of single-LLM approaches by integrating multiple collaborating LLMs (referred to as *agents*) to solve complex code generation tasks [23]. This is achieved by decomposing the overall task into role-specific sub-goals and delegating each sub-goal to a specialized agent, enabling coordinated problem-solving through division of labor (DoL) [24].

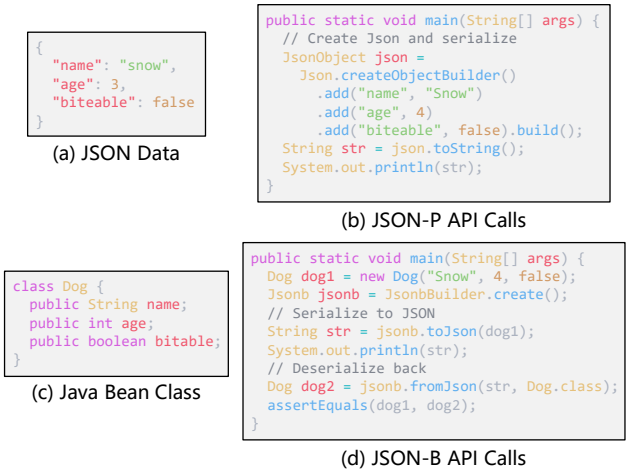

Fig. 1. Examples of JSON-P and JSON-B APIs

B. Automatic Test Generation

Automatic test generation (ATG) techniques aim to automatically create tests for a given software library. The output of an ATG approach is typically a *test suite* that consists of multiple *test cases*. Each generated test case has a *prefix* that sets the conditions necessary to execute the library API method, and a *oracle* (i.e., assertions) that describes the method's expected behavior [7]. By exploring diverse combinations of execution paths, running configurations, and test inputs, the ATG approaches comprehensively exercise the software under test to uncover potential bugs, verify functional correctness, and improve code coverage.

Traditional ATG approaches often employ random algorithms [25] or search-based strategies [26] to synthesize test cases, or adopt model-based methods to accomplish comprehensive functional coverage [27]. More recent research has explored the use of deep learning (DL) to improve ATG [28], [29]. The emergence of LLM has further advanced this area, enabling ATG through prompting to code language models [7], [9], [8].

C. Java JSON Library

JSON (JavaScript Object Notation) is a lightweight, text-based data interchange format. It was formally standardized as ECMA-404 [30] and later by RFC 8259 [31], which defines its syntax and semantic rules, such as allowed data types, string encoding, and number formatting.

Java developers often use JSON for data exchange across various services and applications. To facilitate JSON serialization and deserialization, the Java ecosystem has a wide range of JSON libraries. Among them, *fastjson* [32], *Gson* [33], and *Jackson* [34] are some of the most widely used third-party libraries. They provide rich functionality for parsing JSON strings into Java objects and vice versa, handling common

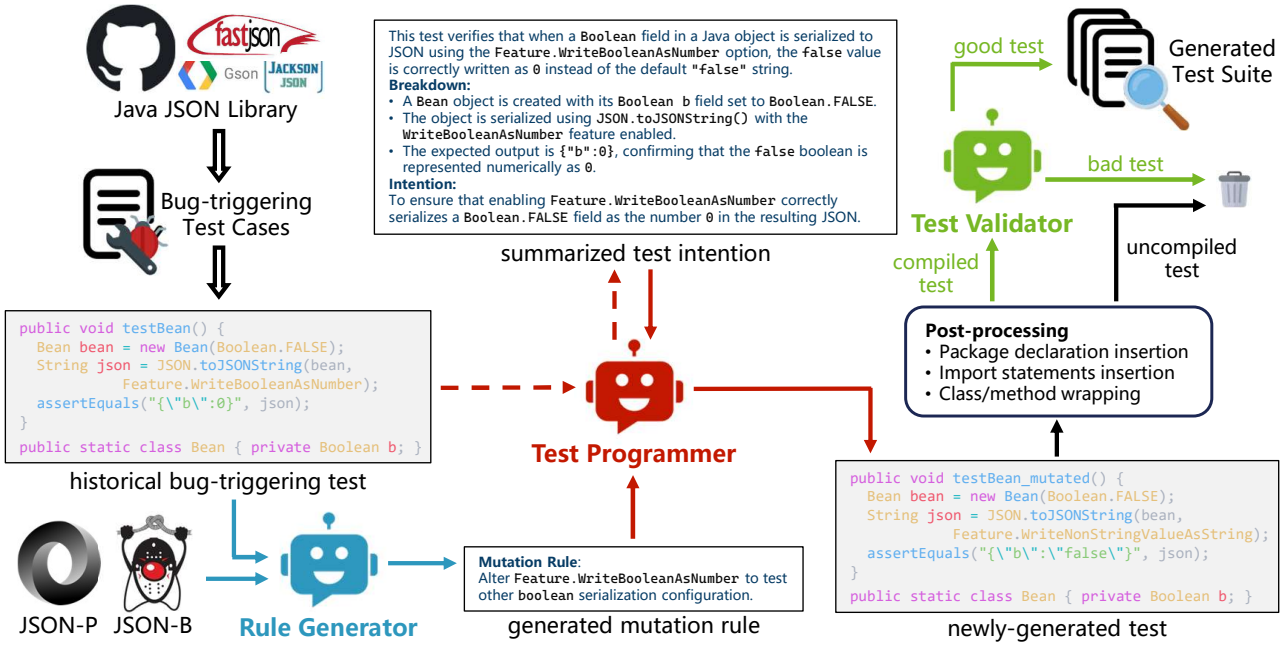

Fig. 2. Overview of JSONATG

data types, nested structures, and custom serialization configuration. However, their implementations (API names, usages, compliance with formal standards, etc.) often diverge in subtle ways. To bring consistency to JSON handling in Java, two official standards were introduced in the Java community:

- **JSON-P** (Java API for JSON Processing) provides low-level streaming and object-model APIs for parsing, generating, and manipulating JSON data. It serves as the foundational reference for JSON parsers in the Java ecosystem.
- **JSON-B** (Java API for JSON Binding) offers a high-level, annotation-driven framework for binding native Java Bean [35] objects to and from JSON data. By standardizing object conversion behaviors, JSON-B enables more convenient and consistent data handling within the Java platform.

Fig. 1 shows examples of JSON-P and JSON-B API usages. These standards serve as reference implementations of Java JSON libraries. Due to the flexibility of third-party libraries, they do not fully conform to the official standards. However, these standards demonstrate general patterns in using JSON data format with the Java language, which allow us to design a generalizable ATG approach among the Java JSON libraries.

III. JSONATG

Fig. 2 illustrates the technical overview of our proposed approach, JSONATG. JSONATG relies on two external resources: the repository of a Java JSON library and the documentation of the Java JSON API standards, namely JSON-P and JSON-B. It incorporates three LLM-based agents, each named to reflect its specific role within the overall ATG workflow: The **Rule Generator** takes as input a historical bug-triggering test along with the relevant JSON-P or JSON-B API documentation, and produces a mutation rule written in natural language. The **Test Programmer** first summarizes the intention of the historical test, then uses this summary with the agent-written mutation rule to generate a new test case. This new test modifies the original test program according to the rule's description. The newly-generated test is then post-processed to correct any syntactic errors and is passed to the **Test Validator**, who assesses its logical validity. The final output of JSONATG is the set of logically valid tests, which we refer to as *good tests* (see Section III-C). The remainder of this section details each agent's design.

A. The Mutation Rule Generator Agent

Effective mutation rules help expose domain-specific bugs in the library. For instance, YANHUI [13] employs prompt-based mutation rules derived from error-prone APIs, data types, and values. However, YANHUI's mutation rules are manually summarized, which limits their generalizability and scalability in the new domain. Instead, given the reference implementation of Java JSON libraries (JSON-P and JSON-B) that describe their common functionalities, we assign the task of generating mutation rules to a dedicated agent. This enables the automatic generation of diverse and potentially unbounded mutation rules tailored to the JSON domain.

The Mutation Rule Generator agent is responsible for writing mutation rules from existing test cases to generate new, semantically meaningful tests. Fig. 3 shows the prompting context for generating the mutation rule. The process is divided into two consecutive LLM queries.

```
System: You are a mutation rule generator.
User: Here is a historical bug-triggering test in
<library name>:
<historical bug-triggering test>
Here is a list of class names from the JSON-P and
JSON-B standards:
<class names in standards>
Please find the most relevant classes of the test
from the given list.
Assistant: <list of the relevant classes>
User: There is the documentation of the classes:
<documentation relevant classes>
According to the test and documentation, write a
mutation rule to mutate the test code.
Here are some examples of mutation rules:
<mutation rule examples>
Assistant: <LLM-generated mutation rule>
```

Fig. 3. Context of the Mutation Rule Generator agent

In the first LLM query, we provide the agent with the historical bug-triggering test case to be mutated, along with a list of API classes from the JSON-P and JSON-B standards. Recall that JSON-P and JSON-B are two official Java API standards that exemplify typical usage patterns for JSON processing in Java, capturing common functionalities that are broadly applicable across different Java JSON libraries. Specifically, JSON-P includes 31 classes, while JSON-B has 26 classes. The agent is instructed to identify the most relevant classes from these standard classes that are associated with the given test case. For the motivating example test illustrated in Fig. 2, the LLM identifies `JsonWriter` from JSON-P and `JsonbSerializer` from JSON-B as the most relevant classes, as both are related to the serialization functionality commonly used in JSON processing.

In the second query, we extract the documentation of the identified relevant classes and prompt the agent to generate a possible mutation rule based on the provided descriptions. To further constrain the output format, we include several example mutation rules as in-context demonstrations [20] for the LLM. These examples include:

- **Extra Data Retrieval**: add additional getter calls and assert the returned value to validate data integrity.
- **Alter Deserialization**: change deserialization method to cover overloaded APIs, such as replacing the string input with a byte array argument.
- **Configuration Option**: switch the method argument to test different serialization or deserialization configurations.
- **Bean Modification**: change field names and types, add or remove fields, or modify nested classes in the bean class.

While syntax-level mutation [36] is feasible, it is hard to adjust the corresponding assertions. LLM can capture dependencies between program elements and thus implicitly handle the adjustment of test oracle [37]. Therefore, instead of applying mutation rules at the syntax level, we will ask the LLM (i.e., the Test Programmer agent) to directly perform program mutation on the original test case.

```
System: You are a test programmer.
User: Here is a historical bug-triggering test in
<library name>:
<historical bug-triggering test>
Please summarize the intention of this test.
Assistant: <LLM-summarized test intention>
User: Based on the test and summarized intention,
write a new test that preserves the same intent
but incorporates the following mutation rule
<agent-generated mutation rule>
Assistant: <LLM-generated new test>
```

Fig. 4. Context of the Test Programmer agent

B. The Test Programmer Agent

Using the same historical bug-triggering test as the Rule Generator, the Test Programmer is responsible for generating a new test case. Its task comprises two phases: 1) summarizing the intention of the original test (illustrated by the dashed line in Fig. 2); and 2) generating a new test that preserves the original intention but introduces slight modifications based on the provided mutation rule (illustrated by the solid line in Fig. 2). The context of the Test Programmer is shown in Fig. 4.

The Test Programmer first takes the original test as input and is prompted to summarize its underlying intention, such as identifying the targeted API method and describing the key operations taken. This summarized intention serves as in-context information to guide the Test Programmer in understanding the original test, as suggested by [38] and [16]. An illustrative example is shown in Fig. 2, where the original test program is decomposed into smaller logical steps. The agent also identifies that the test focuses on the serialization configuration option `Feature.WriteBooleanAsNumber`. Then, with the generated mutation rule that suggests altering this configuration, the Test Programmer replaces the original configuration with `Feature.WriteNonStringValueAsString`. Crucially, with the reasoning capabilities of LLM, the assertion statement is also adapted to reflect the new expected behavior. As shown in the example, the serialized value associated with the key `"b"` changes from the number `0` to the string `"false"`, which correctly corresponds to the altered setting.

In practice, we observed that the agent-generated test code often contains minor errors that prevent successful compilation. To mitigate this issue, we introduce a post-processing phase comprising three key fixing strategies:

1) **Package declaration insertion**: Many generated tests omit the Java package declaration. To fix this, we automatically insert an appropriate package statement based on the project structure of the target Java JSON library.
2) **Import Statement Insertion**: Missing import statements—especially for commonly used classes like assertion utilities—can cause compilation failures. We address this by performing static analysis and inserting necessary import statements to fix all unresolved symbols.
3) **Class and Method Wrapping**: Generated code snippets often lack a surrounding class or method definition. To

```
Bean bean = new Bean(Boolean.FALSE);
String json = JSON.toJSONString(bean,
        Feature.WriteNonStringValueAsString);
assertEquals("{\"b\":false}", json);
```

Fig. 5. A bad test modified from the motivating example

address this, we wrap each snippet in a properly declared test class and method, ensuring it adheres to the Java syntax and testing framework (e.g., JUnit) conventions.

C. The Test Validator Agent

A failed LLM-generated test does not necessarily indicate a fault in the program under test. In many cases, failures may come from logically incorrect test code itself [9]. While the former is desirable for uncovering unknown bugs, the latter can mislead developers and waste their debugging efforts. To differentiate between these scenarios, we categorize test results into two mutually exclusive types:

Definition 1 (Good Test)**.** *A good test fails (at an assertion or by throwing an uncaught exception) due to intrinsic bugs in the library. The logic of a good test is considered correct.*

Definition 2 (Bad Test)**.** *A bad test fails as a result of incorrect test logic or misuse of the library, suggesting that the library works as expected and the test code is faulty.*

The motivating example demonstrated in Fig. 2 is a good test as it causes a test failure and reveals a real bug in *fastjson* (see Section V-C). Fig. 5 shows a bad test modified from this good test. It preserves the test prefix, while the test assertion's expected output is changed: the "false" value is changed from a string into a boolean literal. This modification leads to a logical error because the expected output no longer aligns with the behavior of the `WriteNonStringValueAsString` feature. As a result, the test fails not due to a defect in the program under test, but because of an incorrect test oracle.

For each failed test case, the Test Validator will classify it as either a good test or a bad test. To improve prediction accuracy, we adopt both few-shot and chain-of-thought prompting techniques [39], [21]. Following the idea of [40], the Test Validator executes three independent LLM queries and determines the final label via majority voting. The complete prompt template is shown in Fig. 6. Good test examples are collected from the issue tracking systems of the corresponding Java JSON library. In contrast, bad tests are sampled from LLM-generated failed tests which validated through manual inspection. In considering the context length limitations of the underlying LLM, each prompt includes two good tests and two bad tests.

```
System: You are a test validator.
User: I have test cases for <library name> that
failed or produced errors. We categorize these
tests into two types based on the definitions:
<definitions>
Here are some examples:
[example 1]
JSONObject obj = JSON.parseObject(
                    "{\"V\": -2.0089457e-15}");
assertEquals(-2.0089457e-15, obj.get("V"));
// failed here
assertEquals(-2.0089457e-15, obj.getString("V"));
[result 1]
Test result: FAILURE! AssertionFailedError:
expected: java.lang.Double@3366a998<-2.0089457E
-15> but was: java.lang.String@3c0403a<-2.0089457
E-15>
[answer 1 (CoT)]
The test reports a failure because the assertion
is incorrect, expecting a Double but getting a
String. The revelant code is 'JSON.parseObject
("{\"V\": -2.0089457e-15}").getString("V")'. The
code use 'getString' method to get the value of
key 'V' as String, but expects a Double. The code
 logic is incorrect, and it should be fixed to
use 'getDouble'. Therefore, this is a bad test.
[answer 1 (No CoT)]
The code logic is incorrect. So it is a bad test.
remaining <example, result, answer>
I will give you a test case; your task is to
assess and judge whether the test case is a good
test or a bad test:
<test case to be judged>
Assistant: <answer of LLM>
```

Fig. 6. Prompt template of the Test Validator agent

IV. Experimental Setup

A. Subject Library

Fastjson is a popular high-performance Java JSON library. The upgraded version of *fastjson* is *fastjson2*, designed to address performance and security issues while retaining most of the original API names for backward compatibility. We chose to experiment with *fastjson2* for three reasons:

1) It is a popular Java JSON library that attracts 4.1K stars on GitHub and 8.4K dependent artifacts on Maven Central;
2) It is actively maintained, with over 2K issues on GitHub, many of which have led to the inclusion of regression unit tests (i.e., classes with naming pattern `Issue\d+`);
3) The backward compatibility between *fastjson* and *fastjson2* enables the construction of an extra differential oracle for the generated test cases. However, we use this oracle solely to verify detected bugs for ethical considerations [41].

For simplicity, unless otherwise noted, we refer to *fastjson2* as *fastjson*. We conducted experiments using *fastjson* version 2.0.49, which was the latest release available at the time.

B. Bug-triggering Test Collection

Instead of collecting issues or pull requests (PRs) from the *fastjson* repository, we extracted bug-triggering tests directly from its unit test suite. These tests are typically concise yet contain the essential steps to reproduce specific bugs in the library's APIs. As they have been reviewed and maintained by the project's contributors, they are generally more reliable.

To identify tests related to historical bugs, we used regular expressions to select Java test files whose filenames reference GitHub issues (e.g., test class named `Issue100` corresponds

to *fastjson*'s issue 100). This approach yielded 681 historically bug-triggering unit tests, which we used as the experimental dataset for evaluating JSONATG.

C. Research Questions

Our experiment aimed to answer the following research questions (RQs) regarding JSONATG's performance:

- **RQ1:** How does JSONATG perform compared to the state-of-the-art LLM-based ATG tools in terms of code coverage?
- **RQ2:** How do different components contribute to the overall performance of JSONATG?
- **RQ3:** What bugs in *fastjson* can JSONATG discover?
- **RQ4:** What are the main causes of bad tests in the generated tests, and how effectively can Test Validator classify them?

D. Environments

Our experiments were conducted on a 64-core workstation running Ubuntu 20.04.6 LTS with two NVIDIA Quadro RTX 6000 GPUs, using OpenJDK 1.8.0-382 and Junit 5.11.

E. LLM Selection for Agents

We used four representative LLMs for the agents in JSONATG for our experiments. They included commercial and open-source LLMs:

- **DeepSeek** open-sourced a series of coding LLMs that achieve competitive performance among open-source models. We adopted the 6.7b instruction-tuned model.
- **Llama3** is a state-of-the-art open-source LLM family. We used the instruction-tuned version with 8b parameters.
- **GPT-3.5** is a proprietary LLM offered by OpenAI, and it is the default model used in the early ChatGPT product. We accessed GPT-3.5 Turbo through API services.
- **GPT-4o** is a leading commercial model released by OpenAI, recognized for its strong performance on complex and multi-step reasoning tasks.

For the Mutation Rule Generator, we adopted DeepSeek as the underlying LLM due to its balance between performance and cost-effectiveness. For the Test Validator, we employed GPT-4o, selected for its superior reasoning capabilities, which are essential for its binary classification task. The base model used for the Test Programmer varies according to the RQs under investigation. In RQ1, we used GPT-3.5 to align the configuration with the two baseline approaches. In RQ2, we explored the impact of different LLMs on the Test Programmer to assess their contribution to the overall performance of JSONATG. In RQ3, we extensively ran the Test Programmer with GPT-3.5 to evaluate the cost of bug detection.

F. Baseline Methods

For comparison, we chose two state-of-the-art LLM-based ATG approaches as baseline methods:

- **CHATTESTER** [7] is a ChatGPT-based ATG approach. It consists of a test generator that understands the focal method to produce initial tests and a test refiner that iteratively repairs errors using compiler messages and code context.
- **CHATUNITEST** [9] is another LLM-based ATG framework that demonstrates reliable coverage across diverse projects. It optimizes the LLM through prompt design, adaptive focal context, and a generation-validation-repair mechanism.

There are existing benchmarks for performance testing for Java JSON libraries, such as *JSONTestSuite* [42] and *Java JSON Benchmark* [43]. However, these benchmarks primarily focus on parsing correctness and throughput, rather than on functional correctness, robustness, or bug-finding capabilities. Therefore, they are not well-suited for evaluating ATG tools.

G. Evaluation Metrics

We evaluated JSONATG using two widely adopted metrics: code coverage and the number of newly detected bugs. Code coverage was measured using JaCoCo [44] with the instruction-level granularity. As mentioned in Section IV-A, each test was executed on both *fastjson* and *fastjson2* to verify consistency. Since all seed tests pass on the latest version of *fastjson2*, any newly generated test that fails only on *fastjson2* is considered to reveal a potential bug. We would review such tests and submit bug reports to *fastjson*'s maintainers to determine whether they uncovered previously unknown bugs.

V. RESULTS

A. RQ1: Performance Comparison

1) Method: We used coverage scores to compare the performance of JSONATG against two state-of-the-art approaches. To better assess the ability of ATG approaches to exercise critical components of the target library, we collected coverage for six core classes in *fastjson*: `JSON`, `JSONPath`, `JSONArray`, `JSONObject`, `JSONReader`, and `JSONWriter`.

To run JSONATG, we employed GPT-3.5 as the underlying LLM for the Test Programmer and instructed it to generate three new test cases for each historical bug-triggering test in our dataset. For CHATTESTER and CHATUNITEST, we used their open-source implementations from the Maven plugin [45], with both tools configured to use GPT-3.5 as the underlying LLM. For each test in our dataset, we extracted the respective focal method and instructed the tools to generate three new test cases. We allowed up to three rounds of repair, keeping all other parameters at their default values.

2) Result: As shown in Table I, CHATUNITEST achieved higher coverage for `JSONObject` and `JSONArray`, whereas CHATTESTER had higher coverage for `JSONObject`. This can be attributed to the fact that many methods in both `JSONObject` and `JSONArray` are simple getter methods. Such methods enable CHATUNITEST and CHATTESTER to easily generate valid test cases that invoke them on initialized instances. Although these tests contribute to higher coverage, they only retrieve data immediately after object construction and fail to exercise the library's deeper or more complex functionalities. In contrast, tests generated by JSONATG may not include getters of all data types, resulting in lower coverage scores for these classes.

JSONATG achieved higher coverage scores for four classes: `JSON`, `JSONPath`, `JSONReader`, and `JSONWriter`. In

TABLE I
COMPARISON WITH LLM-BASED APPROACHES CHATTESTER AND CHATUNITEST

Class	Instruction Coverage (%)		
	CHATTESTER	CHATUNITEST	JSONATG
JSON	2.39	6.50	**32.76**
JSONPath	0.33	11.16	**20.46**
JSONArray	27.91	**52.51**	29.64
JSONObject	43.94	**47.47**	41.33
JSONReader	1.88	4.85	**44.86**
JSONWriter	2.68	4.64	**41.88**

contrast, CHATUNITEST and CHATTESTER struggled to cover these larger classes effectively. This limitation stems from their requirement to include the source code of the focal method in the prompt, which often exceeds the maximum token limit. Additionally, they face challenges in generating tests for methods with complex usage scenarios, such as the overloaded parse methods with various arguments in JSON and the more intricate methods in JSONPath. Consequently, they failed to produce valid tests for these classes.

Answer to RQ1. JSONATG achieves higher coverage on complex and large classes in the Java JSON library, demonstrating superior capability in exercising critical components of the library under test. In contrast, state-of-the-art tools struggle with handling complex API usages and prompt length limitations.

B. RQ2: Ablation Study

1) Effectiveness of Mutation Rules: We first evaluate the effectiveness of applying agent-generated mutation rules. For comparison, we include a free-mutation variant [17], in which the Test Programmer is instructed to freely modify the given test without guidance. Following the setup in Section V-A, three mutated tests were generated for each original test.

The results are presented in Table II. We observed higher coverage for tests generated by free mutation in four core *fastjson* classes. This is because more tests generated by agent-written mutation rules fail to compile. As a result, free mutation produces more valid unit tests, resulting in higher coverage. Though provided with instructions and examples along the mutation rules, the LLM may not use them correctly in the generated code, resulting in fewer compiled tests.

Prompting with free mutation and agent-written mutation rules can both identify bugs, as seen in Fig. 7. We observed that both mutation rules (either free or agent-written) are capable of identifying unique bugs, with three bugs overlapping in between. Additionally, five unique bugs were identified using free mutation, while four unique ones were identified using agent-written mutation. We found that LLM tends to make more conservative changes to the original tests with free mutation, resulting in the detection of certain unique bugs. **However, they were mostly known bugs as confirmed by the developers.** In contrast, the LLM produced larger changes with agent-written mutation, thereby discovering additional

TABLE II
COVERAGE SCORES OF APPLYING DIFFERENT MUTATION RULES

Class	Instruction Coverage (%)	
	Free Mutation	Agent-written Mutation
JSON	**36.14**	32.76
JSONPath	**28.17**	20.46
JSONArray	26.35	**29.64**
JSONObject	37.53	**41.33**
JSONReader	**47.10**	44.86
JSONWriter	**43.74**	41.88

Fig. 7. Detected bugs of applying different mutation rules

distinct bugs. This suggests that incorporating such rules into the prompts can help uncover distinct and new bugs that the base setting alone might miss. In conclusion, applying agent-written mutation rules helps diversify the generated test suite.

2) Effectiveness of Summarizing Test Intention: To assess the impact of integrating test intention summarization into the Test Programmer's context, we instructed JSONATG to generate three new tests for each bug-triggering test in our dataset, this time omitting the test summarization step. To control uncertainty, we constrained the Test Programmer to apply only the free mutation rule. We then compared the resulting test suite with that obtained in Section V-B1.

TABLE III
TEST EXECUTION RESULT OF TEST SUMMARIZATION (%)

Execution Result	w/ summarization	w/o summarization
Pass	56.3	54.2 ↓
Failure/Exception	29.8	20.1 ↓
Compile Error	13.9	25.7 ↑

Table III shows that tests generated with intention summarization achieve a higher pass rate (56.3% vs. 54.2%) compared to those generated without summarization. Notably, removing the summarization step leads to a significant increase in compilation errors, rising from 13.9% to 25.7%. Our in-depth investigation revealed that these compilation failures were primarily caused by missing bean class definitions (70%), missing import statements (20%), and hallucinated method names (10%). Although the failure rate of executable tests is slightly higher with summarized intentions (29.8% vs. 20.1%), both passing and failing tests contribute to code coverage and bug detection. Overall, incorporating intention summarization produces test suites that are more practically useful.

3) Performance Variation of Different LLMs: To evaluate the effectiveness of Test Programmer with different LLMs, we experimented with open-source LLMs Llama3 and DeepSeek, and the commercial LLM GPT-3.5. All models were configured with a *top-p* of 0.95 and a *temperature* of 0.8. We also

TABLE IV
COVERAGE SCORES OF DIFFERENT LLMS FOR THE TEST PROGRAMMER

Class	Instruction Coverage (%)		
	Llama3	DeepSeek	GPT-3.5
JSON	28.19	36.12	**36.14**
JSONPath	19.67	8.22	**28.17**
JSONArray	22.39	22.81	**26.35**
JSONObject	30.23	34.95	**37.53**
JSONReader	40.15	43.90	**47.10**
JSONWriter	36.24	41.88	**43.74**

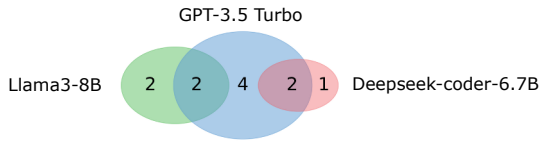

Fig. 8. Detected bugs of different LLMs for the Test Programmer

prompted them to perform free mutations. We asked each LLM to generate three tests for each of the original bug-triggering tests and compare their resulting test suite.

The coverage scores of the generated tests are shown in Table IV. Among the three models, Llama3 achieved the lowest coverage score. The reason is that it suffers from a higher compile error rate of 25.71%, compared to 13.82% for GPT-3.5 and 12.73% for DeepSeek. GPT-3.5 achieved the highest coverage, reflecting its superior capabilities to generate accurate and functional code across diverse scenarios. DeepSeek follows closely behind GPT-3.5, showing a competitive performance in generating code with fewer compile errors.

Fig. 8 shows the overlaps between the identified bugs triggered by different LLMs. Overall, GPT-3.5 identified the most bugs among the models, while both Llama3 and DeepSeek detected unique bugs that were not triggered by GPT-3.5. While LLMs vary in overall performance, differences in their training data and methodologies enable them to identify distinct, non-overlapping functional defects. This suggests that, in practice, combining outputs of different models can enhance bug detection and lead to more comprehensive testing.

Answer to RQ2. Agent-written mutation rules help uncover unique bugs that the base prompt may miss. Involving test summarization reduces the compile error rate. Additionally, the Test Programmer can be adapted to use different LLMs to detect more diverse bugs.

C. RQ3: Detected Bugs

We allocated the OpenAI API budget to $25 and ran JSONATG to extensively test the *fastjson* library, consuming approximately 40 CPU hours. Within this budget, JSONATG revealed 59 bugs in *fastjson* that relate to various features, such as annotations, `JSONPath` and (de)serialization. A complete list of our reported bugs can be found in *fastjson*'s issue tracking system[1]. Among the reported bugs, 47 were confirmed by *fastjson*'s developers, and 28 had already been fixed in the latest release. Here, we present three representative examples of the discovered bugs to demonstrate the practical usefulness of JSONATG.

```
1 Bean bean = new Bean(Boolean.FALSE);
2 String json = JSON.toJSONString(bean,
3           Feature.WriteNonStringValueAsString);
4 assertEquals("{\"b\":\"false\"}", json);
5
6 public static class Bean { private Boolean b; }
```

Listing 1. A serialization bug with `Boolean` wrapper class

Listing 1 illustrates a serialization bug involving the `Boolean` wrapper class, as previously shown in Fig. 2. The `JSON.toJSONString` method is invoked to serialize an object, where non-string values should be written as strings (i.e., enclosed in quotation marks). However, *fastjson* produces the incorrect output `"{\"b\":false}"` at line 4, omitting the quotation marks around the `Boolean` value and thereby causing an assertion failure. This bug was confirmed and discussed in issue 2560. Listing 1 is generated from the test in issue 1874. The Test Programmer, guided by the agent-generated mutation rule, replaces the original configuration option, successfully revealing a serialization bug.

```
1 JSONObject obj = JSONObject.of("data",
2                                JSONArray.of(1));
3 String str = JSON.toJSONString(obj);
4 assertEquals(JSONPath.eval(str, "$.data[0][0]"),
5              JSONPath.eval(obj, "$.data[0][0]"));
```

Listing 2. A bug in the `JSONPath` class

Listing 2 demonstrates an example of an inconsistency bug in `JSONPath`. The `.eval()` method produces different results when processing `JSONObject` and `String`, even though they contain the same JSON data. This case leads to a failed assertion at line 4. The bug was confirmed in issue 2584. The original test behind Listing 2 can be found in issue 1965. It only checks for null values of the `JSONPath`'s evaluation results. With this test case, JSONATG's Test Programmer made a valid modification that directly compares the evaluation results on both the original `JSONObject` and the serialized string, which successfully triggers an inconsistency bug.

```
1 BigDecimal decimal = BigDecimal.valueOf(
2            Long.MAX_VALUE).add(BigDecimal.ONE);
3 String str = JSON.toJSONString(decimal);
4 assertEquals("9223372036854775808", str);
5 BigDecimal dec1 = (BigDecimal)JSON.parseObject(
6                         str, BigDecimal.class);
7 assertEquals(decimal.stripTrailingZeros(), dec1);
```

Listing 3. A deserialization bug with `BigDecimal` class

Listing 3 shows a deserialization bug with the JDK class `BigDecimal`. When attempting to parse a JSON string into a `BigDecimal`, the `parseObject` method incorrectly interprets it as negative, returning -9223372036854775808 and triggering an assertion failure at line 7. This bug was

[1]https://www.github.com/alibaba/fastjson2/issues?q=is%3Aissue%20state%3Aclosed%20author%3ACooperzzy

confirmed in issue 2582. The original test case behind this bug appears in issue 1204. In this case, the Test Generator uses the `parseObject` method instead of the original `parse`, thereby uncovering a bug related to numeric overflow.

By comparing the differences between the original and newly generated tests, we conclude that JSONATG effectively enhances the diversity of historical bug-triggering test cases, thereby facilitating the discovery of new bugs.

Answer to RQ3. JSONATG effectively facilitates the discovery of 59 reported bugs in *fastjson*, of which 47 were confirmed and 28 have already been fixed.

D. RQ4: Failures Analysis

1) Failures of Generated Tests: In Section V-A, among all of the JSONATG-generated tests that compiled, 67.0% passed, 25.6% failed due to assertion mismatches, and 7.4% crashed due to other exceptions or errors. We inspected the failing and crashing tests to identify **if they were bad tests** and investigate their root causes.

For tests that fail at assertions, the most common cause is the assertion of contradictory expected values. Typical scenarios of such failing test assertions include: 1) expecting field names that contradict the bean fields; 2) wrongly asserting a null or non-null object; or 3) asserting incorrect data types. Another common issue is incorrect expected values of the serialized string, such as the one shown in Fig. 5.

The top three exceptions thrown are: `JSONException`, `NullPointerException`, and `ClassCastException`. The primary cause is that parsing an incorrect JSON string, either syntactically incorrect or mismatched with the bean definition, directly results in a library-defined `JSONException`. Other reasons include calling parsing/serializing functions with incorrect parameters, casting data to incompatible types, and invoking methods on a null object that was retrieved from a non-existent key or index.

2) Bad Test Classification by Test Validator: Our investigation above confirmed that the test cases generated by the Test Programmer do include bad tests, highlighting the need for the Test Validator agent in the overall framework. Therefore, this subsection evaluates the classification accuracy of the Test Validator in isolation. We collected 21 bad tests and 22 good tests. Among them, 10 bad tests triggered exceptions (E_{bad}), 11 bad tests failed at assertions (F_{bad}), 10 good tests triggered exceptions (E_{good}), and 12 good tests failed at assertions (F_{good}). Instances of E_{good} and F_{good} were collected from test cases that have identified bugs, and we sampled a similar number of E_{bad} and F_{bad} from the generated tests to ensure balanced sample sizes. We then ran Test Validator using the prompt in Fig. 6 and recorded the results, as shown in Table V.

For bad test samples that fail due to assertion mismatches (F_{bad} and F_{good}), the Test Validator demonstrates a stronger ability to identify them. However, for cases that throw exceptions (E_{bad} and E_{good}), its performance appears closer to random guessing. This can be attributed to LLM's ability to reason about code behavior, while its difficulty lies in predicting library-defined exceptions (e.g., `JSONException` in *fastjson*). Such exceptions can be triggered by both invalid test logics (bad tests) and internal library issues (good tests), making accurate classification more challenging. Interestingly, the few-shot with chain-of-thought (FS-CoT) approach, which includes reasoning steps in the few-shot examples, performs similarly to the basic few-shot (FS) method.

TABLE V
BAD TEST CLASSIFICATION RESULTS

	E_{bad}	E_{good}	F_{bad}	F_{good}	Accuracy (avg.)
FS	7/10	4/10	10/11	10/12	72.1%
FS-CoT	5/10	5/10	10/11	10/12	69.8%

Among the 14 misclassified instances by both FS and FS-CoT, 11 are shared between the two approaches. Many of these misclassifications are related to specific behaviors of *fastjson*, which are inherently difficult to judge even for experienced users [46]. As such, even with explicit reasoning steps, accurately assessing new cases remains challenging without prior domain-specific knowledge. Nevertheless, the experimental results indicate that the Test Validator is still able to identify a meaningful portion of bad tests, contributing to the quality improvement of JSONATG's generated test suites.

Answer to RQ4. Most bad tests come from assertion mismatches and library-defined exceptions. While Test Validator effectively identifies bad tests that fail at assertions, it struggles to accurately classify those that trigger library-defined exceptions.

VI. DISCUSSIONS

A. Lessons Learned

• **Existing general-purpose LLM-based ATG approaches are better at generating regression test suites than bug-triggering tests.** In our experiment, most test cases produced by CHATTESTER and CHATUNITEST are short and trivial. They generally consist of straightforward method invocations and object initializations without complex data manipulation operations. Such tests can easily achieve a promising coverage score and thus are suitable for regression purposes. However, these tests rarely uncover edge cases or unusual input, limiting their practical use for bug finding.

• **Leveraging different LLMs can enhance the diversity of generated tests, thereby improving the bug detection capability.** By using different LLMs, such as open-source (Llama3), coding-specific (DeepSeek-coder), and commercial (GPT-3.5) LLMs, JSONATG uncovers a broader range of bugs. Differences in pre-training and fine-tuning phases lead to varied generation styles for different models, contributing to diverse generated tests. Notably, in our experiment, a smaller model (with 6.7B parameters) can also perform well in generating bug-triggering tests for *fastjson*, comparable to larger commercial models like GPT-3.5.

• **Explicitly importing expert knowledge enhances LLM-based ATG in specialized domains.** Throughout our research,

we found that providing domain-specific context—such as curated API documentation, known usage patterns, and historical bug-triggering tests—can improve the relevance and correctness of the generated test cases. Rather than relying solely on the general-purpose capabilities of the LLM, encoding expert knowledge into the prompt or system design guided the model to produce more meaningful and valid outputs. This is particularly important for specialized libraries, where correct usages often depend on complex serialization behaviors that are not easily inferred from natural language alone. Our results suggest that LLM-based tools for automated testing can benefit greatly from structured expert input, especially in domains with non-trivial semantics or specialized APIs.

B. Limitations and Future Work

• **Uncompiled and bad generated test cases.** JSONATG may produce test cases that fail to compile, which wastes computational resources and potentially loses otherwise valuable tests. To address this issue, we apply a series of lightweight post-processing steps to the outputs of JSONATG's Test Programmer. Despite these efforts, a significant number of test cases remain uncompiled and are discarded. One potential solution is to incorporate more fine-grained domain knowledge (e.g., valid examples of Java JSON library usages) into the prompts, which may help reduce such errors [13]. Similarly, test cases labeled as "bad tests" by the Test Validator are also discarded. For these, applying automatic program repair techniques—such as LLM-based self-debugging—offers a promising direction for reusing the discarded tests [15], [16].
• **False positives of Test Validator's prediction.** The generated tests may contain logical errors; therefore, we introduce the Test Validator to identify and filter out such bad test cases. However, as shown in our experiments, the Test Validator may produce false positives. To support practical bug detection, we employed a differential oracle [47] between the *fastjson2* library and its predecessor, *fastjson*, to isolate true bugs. However, this approach requires multiple implementations of the same specification, which may not hold in general development scenarios. As future work, instead of relying on the Test Validator, we plan to explore test migration [48] as a way to facilitate differential testing. This would involve comparing semantically equivalent tests across various Java JSON libraries. The challenge here lies in the differences between library APIs—some APIs have direct counterparts in other libraries, while others are specific to a particular library, lacking equivalent functionality elsewhere. A possible direction is to integrate the Test Programmer agent with a reasoning model to form a joint test oracle, further enhancing the testing process and generalizability across different Java JSON libraries.

C. Threats to Validity

The main threat to validity lies in the inherent randomness of the LLM agents' outputs. To evaluate the effectiveness of LLMs, we experimented with both open-source models (Llama3 and DeepSeek) and a commercial model (GPT-3.5) for the Test Programmer agent. To ensure stable output, we fix generation parameters such as *temperature* and *top-p* sampling, and use specific versions of each agent's underlying LLM.

In JSONATG's design, we assume that the JSON-P and JSON-B standards apply to the tested Java JSON libraries; however, this assumption may not hold universally across all APIs and libraries. To mitigate this threat, we use these standards as references for common usage patterns rather than as strict API dependencies. Moreover, by incorporating the original test case, JSONATG adapts to the specific usage pattern of the target library, allowing it to remain effective even when the library partially deviates from the standard APIs.

VII. Related Work

A. Traditional ATG Approaches

ATG approaches [49] aim to extensively exercise the software under test by exploring diverse combinations of execution paths, running configurations, test inputs, etc. Existing studies typically addressed the challenges of mitigating combinatorial explosions during software executions. Before the emergence of LLM, traditional approaches often employed random-based [25] or search-based [26] strategies to synthesize test programs, or adopted model-based algorithms to accomplish comprehensive functional coverage [27]. A notable example is EvoSuite [26], which applies an evolutionary algorithm to generate test cases that maximize code coverage. Although achieving good coverage measurement, these approaches often produce tests that lack readability and maintainability, making them difficult for developers to use in practice [6], [50]. More recent research has explored using deep learning techniques to improve ATG [28], [29].

Fuzzing [51] with program libraries often means generating complete programs or fuzzing harnesses that cover various library APIs, which is similar to ATG. RULF [52] generates fuzz targets in Rust libraries by traversing their API dependency graph. Rahalkar presents a system that generates fuzzing harnesses for library APIs and binary protocol parsers by analyzing unit tests [53]. Chen et al. [4] present a general fuzzer for testing libraries without necessitating domain-specific knowledge in creating fuzz drivers.

These techniques have been used for testing JSON libraries. For example, the JSON fuzzer of *fastjson2* in OSS-Fuzz [54] targets a specific parsing method, while EvoGFuzz [55] generates inputs for eight JSON parsers to uncover defects. However, they did not specifically target Java JSON libraries, leaving a wide range of common functionalities untested, and JSONATG fills this gap.

B. LLMs for ATG

The emergence of LLMs has enabled ATG through prompting with natural language. For instance, FuzzGPT [47] and TitanFuzz [12] utilize LLMs to generate Python code for testing deep learning libraries. Fuzz4All [17] is an LLM-based universal fuzzer for multiple programming language infrastructures. Yuan et al. [7] conducted an empirical study on ChatGPT's ATG abilities and introduced the ChatTester

tool. TESTPILOT [8] generates unit tests for JavaScript methods within project APIs. CHATUNITEST [9] provides an LLM-empowered framework with user-friendly APIs for ATG. Meta Inc. uses LLMs to automatically improve existing human-written tests and has been deployed at scale in production [18]. UTGEN [11] combines search-based testing with LLMs to improve the understandability of the generated unit tests. While they have shown promise in generating high-coverage tests, challenges remain in ensuring the correctness of the generated tests, as LLM outputs can be non-deterministic and prone to hallucinations.

Unlike previous single-LLM approaches, JSONATG orchestrates three specialized LLM agents to enable end-to-end ATG. By decomposing the overall task into well-defined subtasks and assigning each to a dedicated agent, JSONATG emulates the workflow of an expert human tester, that is: 1) analyzing historical bug-triggering tests and library specifications to derive meaningful mutation rules; 2) interpreting the intended test behavior and generating new tests guided by the rule; and 3) inspecting the generated tests to assess their validity.

VIII. CONCLUSION

We present JSONATG, a multi-agent ATG system specialized for Java JSON libraries. Leveraging high-quality historical bug-triggering tests and LLM-based agents for mutation rule generation, code summarization, test generation, and validation, JSONATG produces diverse and valid test cases that reveal both crashing and non-crashing bugs in the widely used Java JSON library. Our experiments on *fastjson* show that JSONATG outperforms two state-of-the-art LLM-based ATG tools in terms of code coverage and detected bugs. It also demonstrates practical usefulness by reporting 59 bugs, of which 47 were confirmed and 28 have already been fixed. These results underscore JSONATG's potential to advance automatic quality assurance in the JSON domain.

ACKNOWLEDGMENT

We would like to thank all anonymous reviewers for their valuable comments on this paper. This work is supported by the National Natural Science Foundation of China (Grant No. 62372219).

REFERENCES

[1] N. Harrand, T. Durieux, D. Broman, and B. Baudry, "The behavioral diversity of java json libraries," in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2021, pp. 412–422.

[2] "CVE Record: CVE-2017-18349," https://www.cve.org/CVERecord?id=CVE-2017-18349, 2017.

[3] "CVE Record: CVE-2024-27454," https://www.cve.org/CVERecord?id=CVE-2024-27454, 2024.

[4] P. Chen, Y. Xie, Y. Lyu, Y. Wang, and H. Chen, "Hopper: Interpretative fuzzing for libraries," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1600–1614.

[5] X. Sun, X. Chen, Y. Zhao, P. Liu, J. Grundy, and L. Li, "Mining android api usage to generate unit test cases for pinpointing compatibility issues," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

[6] M. M. Almasi, H. Hemmati, G. Fraser, A. Arcuri, and J. Benefelds, "An industrial evaluation of unit test generation: Finding real faults in a financial application," in *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE, 2017, pp. 263–272.

[7] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, "Evaluating and improving chatgpt for unit test generation," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1703–1726, 2024.

[8] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.

[9] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "Chatunitest: A framework for llm-based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 572–576.

[10] M. L. Siddiq, J. C. Da Silva Santos, R. H. Tanvir, N. Ulfat, F. Al Rifat, and V. Carvalho Lopes, "Using large language models to generate junit tests: An empirical study," in *Proceedings of the 28th international conference on evaluation and assessment in software engineering*, 2024, pp. 313–322.

[11] A. Deljouyi, R. Koohestani, M. Izadi, and A. Zaidman, "Leveraging large language models for enhancing the understandability of generated unit tests," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1449–1461.

[12] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, 2023, pp. 423–435.

[13] H. Guan, G. Bai, and Y. Liu, "Large language models can connect the dots: Exploring model optimization bugs with domain knowledge-aware prompts," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1579–1591.

[14] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[15] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1482–1494.

[16] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=KuPixIqPiq

[17] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[18] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, "Automated unit test improvement using large language models at meta," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 185–196.

[19] D. Xie, Z. Zhang, N. Jiang, X. Xu, L. Tan, and X. Zhang, "Resym: Harnessing llms to recover variable and data structure symbols from stripped binaries," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4554–4568.

[20] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[21] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[22] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: empowering code generation with oss-instruct," in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 52 632–52 657.

[23] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via chatgpt," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 7, pp. 1–38, 2024.

[24] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,"

ACM Transactions on Software Engineering and Methodology, vol. 34, no. 5, pp. 1–30, 2025.

[25] C. Pacheco and M. D. Ernst, "Randoop: feedback-directed random testing for java," in *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, 2007, pp. 815–816.

[26] G. Fraser and A. Arcuri, "Evosuite: automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, 2011, pp. 416–419.

[27] E. P. Enoiu, A. Čaušević, T. J. Ostrand, E. J. Weyuker, D. Sundmark, and P. Pettersson, "Automated test generation using model checking: an industrial evaluation," *International Journal on Software Tools for Technology Transfer*, vol. 18, no. 3, pp. 335–353, 2016.

[28] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri, "Toga: A neural method for test oracle generation," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2130–2141.

[29] C. Watson, M. Tufano, K. Moran, G. Bavota, and D. Poshyvanyk, "On learning meaningful assert statements for unit test cases," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1398–1409.

[30] "ECMA-404 - Ecma International," https://ecma-international.org/publications-and-standards/standards/ecma-404/, 2017.

[31] T. Bray, "RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format," https://datatracker.ietf.org/doc/html/rfc8259, 2017.

[32] "alibaba/fastjson2: FASTJSON2 is a Java JSON library with excellent performance," https://github.com/alibaba/fastjson2.

[33] "google/gson: A Java serialization/deserialization library to convert Java Objects into JSON and back," https://github.com/google/gson.

[34] "FasterXML/jackson: Main Portal page for the Jackson project," https://github.com/FasterXML/jackson.

[35] J. White, D. C. Schmidt, and A. Gokhale, "Simplifying autonomic enterprise java bean applications via model-driven development: A case study," in *International Conference on Model Driven Engineering Languages and Systems*. Springer, 2005, pp. 601–615.

[36] X. Ou, C. Li, Y. Jiang, and C. Xu, "The mutators reloaded: Fuzzing compilers with large language model generated mutation operators," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2024, pp. 298–312.

[37] S. B. Hossain and M. B. Dwyer, "Togll: Correct and strong test oracle generation with llms," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1475–1487.

[38] C. Agarwal, S. H. Tanneru, and H. Lakkaraju, "Faithfulness vs. plausibility: On the (un) reliability of explanations from large language models," *arXiv preprint arXiv:2402.04614*, 2024.

[39] Y. Song, T. Wang, P. Cai, S. K. Mondal, and J. P. Sahoo, "A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–40, 2023.

[40] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=1PL1NIMMrw

[41] S. Wang, Y. Wang, X. Zhan, Y. Wang, Y. Liu, X. Luo, and S.-C. Cheung, "Aper: evolution-aware runtime permission misuse detection for android apps," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 125–137.

[42] "nst/JSONTestSuite: A comprehensive test suite for RFC 8259 compliant JSON parsers," https://github.com/nst/JSONTestSuite.

[43] "fabienrenaud/java-json-benchmark: Performance testing of serialization and deserialization of Java JSON libraries," https://github.com/fabienrenaud/java-json-benchmark.

[44] "jacoco/jacoco: Java Code Coverage Library," https://github.com/jacoco/jacoco.

[45] "ZJU-ACES-ISE/chatunitest-maven-plugin," https://github.com/ZJU-ACES-ISE/chatunitest-maven-plugin.

[46] "Fastjson2 issue 3153," https://github.com/alibaba/fastjson2/issues/3153.

[47] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries," in *Proceedings of the 46th IEEE/ACM international conference on software engineering*, 2024, pp. 1–13.

[48] Y. Gao, X. Hu, T. Xu, X. Xia, D. Lo, and X. Yang, "Mut: Human-in-the-loop unit test migration," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.

[49] D. Serra, G. Grano, F. Palomba, F. Ferrucci, H. C. Gall, and A. Bacchelli, "On the effectiveness of manual and automatic unit test generation: ten years later," in *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 2019, pp. 121–125.

[50] L. Yang, C. Yang, S. Gao, W. Wang, B. Wang, Q. Zhu, X. Chu, J. Zhou, G. Liang, Q. Wang *et al.*, "On the evaluation of large language models in unit test generation," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1607–1619.

[51] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2019.

[52] J. Jiang, H. Xu, and Y. Zhou, "Rulf: Rust library fuzzing via api dependency graph traversal," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 581–592.

[53] C. Rahalkar, "Automated fuzzing harness generation for library apis and binary protocol parsers," *arXiv preprint arXiv:2306.15596*, 2023.

[54] "google/oss-fuzz: OSS-Fuzz - continuous fuzzing for open source software," https://github.com/google/oss-fuzz.

[55] M. Eberlein, Y. Noller, T. Vogel, and L. Grunske, "Evolutionary grammar-based fuzzing," in *International Symposium on Search Based Software Engineering*. Springer, 2020, pp. 105–120.